

PHẦN 1 & 3: BÁO CÁO PHÂN TÍCH VÀ ĐẶC TẢ SRS (TÍCH HỢP THANH TOÁN)

1. Đề xuất và phân tích giải pháp tích hợp thanh toán

- Giải pháp 1: Gọi API đồng bộ (Sync API / Real-time)
Khách hàng thao tác, hệ thống gọi trực tiếp API sang Cổng thanh toán/Ngân hàng và giữ kết nối chờ phản hồi. Nếu thành công, hệ thống cập nhật trạng thái ngay lập tức.
- Giải pháp 2: Bất đồng bộ qua Webhook & Message Queue (Async) Hệ thống sinh mã thanh toán (VietQR) và không chờ kết quả. Khi khách chuyển khoản xong, Ngân hàng đẩy thông báo kết quả (Webhook) vào Message Queue của RikkeiLogistics. Các Worker (Consumer) sẽ lấy message từ hàng đợi ra xử lý dần.

Bảng so sánh Trade-off

Tiêu chí	Giải pháp 1 (Sync API)	Giải pháp 2 (Async Webhook + Queue)
Độ tin cậy khi nghẽn mạng	Rất thấp: Dễ xảy ra tình trạng Timeout. Khách đã bị trừ tiền ở ngân hàng nhưng kết nối đứt ngang khiến RikkeiLogistics không ghi nhận, gây khiếu nại.	Rất cao: Message Queue đóng vai trò vùng đệm. Nếu Billing Service quá tải hoặc sập, tin nhắn thanh toán vẫn lưu trong Queue và được xử lý bù khi hệ thống phục hồi.
Độ phức tạp triển khai	Thấp: Dễ lập trình, luồng logic đi thẳng từ đầu đến cuối trong một hàm xử lý.	Cao: Cần thiết lập hệ thống Queue (Kafka/RabbitMQ), viết Consumer Worker chạy ngầm, cấu hình cơ chế Idempotency Key để chống lưu trùng lặp khi nhận Webhook 2 lần.
Rủi ro sai lệch dữ liệu tài chính	Cao: Nguy cơ mất đồng bộ (Inconsistent state) do giao dịch lơ lửng khi mạng chập chờn. Không đảm bảo tính ACID khi kết nối bị ngắt đột ngột.	Thấp: Đảm bảo tính nhất quán cuối cùng (Eventual Consistency). Các giao dịch được Worker xử lý tuần tự, có thể config retry (thử lại) nếu Database bị lock.

Giải pháp được chọn: Giải pháp 2 (Async Webhook + Queue).

Giải thích: Với quy mô 200.000 giao dịch/ngày, hệ thống đòi hỏi khả năng chịu tải cao (High Throughput). Việc sử dụng Message Queue giúp "hứng" các đợt lưu lượng tăng đột biến (ví dụ: đợt sale lễ Tết) mà không làm sập server. Đồng thời, cơ chế Idempotency Key kết hợp

Queue giải quyết triệt để bài toán bão dữ liệu (Webhook gửi trùng 2 lần) và chống thất thoát dữ liệu khi timeout.

2. Danh sách bảng dữ liệu & Cơ chế bảo mật (Đặc tả SRS)

Danh sách bảng dữ liệu cần thiết (Không vẽ ERD):

1. Shipment: Lưu thông tin vận đơn, thông tin người gửi/nhận, cước phí, số tiền COD cần thu và trạng thái đơn hàng.
2. PaymentTransaction: Lưu lịch sử giao dịch thanh toán cước/COD. Chứa **TransactionID** (từ ngân hàng), **IdempotencyKey** (để chống Webhook đẩy trùng) và trạng thái gạch nợ. Đảm bảo tính ACID.
3. CODSettlement: Lưu thông tin đối soát tiền thu hộ. Chứa thông tin tài khoản ngân hàng/ví của người gửi, số tiền cần chuyển, thời hạn (trong 24h) và trạng thái chuyển tiền.
4. AuditLog: Lưu vết (Audit Trail) toàn bộ các thao tác tài chính, sự kiện thay đổi trạng thái, các luồng Webhook bị từ chối do sai chữ ký số để phục vụ tra soát.

Cơ chế bảo mật giao dịch:

- API Key: Cấp phát một chuỗi định danh duy nhất để định danh Cổng thanh toán/Ngân hàng khi họ gọi Webhook sang hệ thống RikkeiLogistics.
- HMAC Checksum (HMAC-SHA256): Ngân hàng dùng một Secret Key (chỉ 2 bên biết) để băm (hash) toàn bộ nội dung dữ liệu thanh toán gửi đi thành một chuỗi mã hóa (Signature). Khi Billing Service nhận được Webhook, hệ thống dùng Secret Key đó băm lại payload. Nếu hai chuỗi khớp nhau, xác nhận dữ liệu toàn vẹn (không bị can thiệp dọc đường) và đúng nguồn phát; trạng thái đơn hàng mới được phép chuyển sang PAID.

